Safety as a Schedulable Resource: Budget-Aware Admission Control for Multi-Tenant LLM Gateways

Taixing Bi
Independent Researcher
New York, NY, USA
taixingbijob@gmail.com

***Abstract*—Enterprise large language model (LLM) gateways can use a lightweight risk estimator to decide whether an incoming request requires a stronger, more expensive safety check before it reaches a generation model. Existing risk-based routing treats this decision purely as a per-request classification problem: is this request risky enough to warrant a strong guardrail? This framing ignores that the strong guardrail is itself a bounded shared resource, constrained by third-party API cost, per-tenant policy, and organizational safety budgets. We argue that strong safety enforcement should instead be treated as a schedulable resource in multi-tenant LLM gateways, alongside compute and queueing capacity. We present a safety admission controller that jointly considers request risk, tenant policy, available safety budget, queue load, and per-request deadlines, and that enforces a fail-closed invariant: a request whose policy mandates a strong check either receives that check or is rejected early, and is never silently passed through ungated. We characterize the lightweight risk estimator on a frozen, validated benchmark of approximately 1.9K prompts spanning four risk classes, and externally on XSTest and WildGuardTest. Using the resulting frozen risk scores, we then evaluate the admission controller itself across multi-tenant contention, dynamic safety demand, fail-open overload, and deadline-aware rejection, and show that tenant-aware scheduling raises strong-check coverage for a minority sensitive tenant from 29.9% to 65.5% at a 90:10 tenant mix, and lowers that tenant's unsafe admission rate (e.g., 0.242 to 0.175 at 70:30); that safety-resource exhaustion can occur independently of changes in aggregate offered load as the risky-request fraction rises from 5% to 50% under constant total request-per-second (RPS); that at $1.5B_g$ offered demand, fail-open policies bypass 57% of policy-required strong checks and raise the unsafe admission rate from 0.051 to 0.611 for only a 0.017 RPS gain in Safe SLO-Goodput, with bypass reaching 62% across the swept range; and that rejecting a required check at admission when it cannot start in time, rather than queueing it, reduces strong-guardrail calls from 39 to 29 with unchanged Safe SLO-Goodput, while cutting safety-stage P95 latency from the 2 s queue timeout (2002 ms) to 178 ms. These results position guardrail capacity as a first-class scheduling target rather than an afterthought of risk classification.**

***Index Terms*—LLM gateway, admission control, guardrails, multi-tenant systems, safety-resource scheduling, overload control**

I. Introduction

Enterprise deployments of large language models increasingly sit behind a gateway that mediates traffic from many tenants toward one or more backend generation models. To keep latency and cost manageable, a gateway can use a lightweight risk estimator—a small classifier such as a distilled sentence encoder—to score each incoming request and decide whether it warrants a stronger, more expensive safety check, for example a managed guardrail service, before being forwarded to generation [3], [4], [7]. This pattern reduces the question of request safety to a single per-request decision: is this request risky enough to need a strong guardrail?

That framing is incomplete. The strong guardrail is not a free, infinitely available function call. In practice it is a bounded shared resource: it has a cost per invocation, it may be subject to a third-party rate limit, and an organization may impose an explicit budget on how much strong-guardrail traffic it is willing to pay for or route through a given provider in a given window. When many tenants share one gateway, and each tenant may carry a different safety policy and a different sensitivity to risk, the interesting systems question is no longer just "is this request risky" but rather: *when multiple tenants compete for a bounded strong-guardrail budget under dynamic safety demand, how should the gateway allocate safety checks while respecting tenant policy and latency deadlines?*

This paper answers that question by treating strong-guardrail capacity as a first-class, schedulable resource, analogous to how modern serving systems treat GPU or queueing capacity [10]–[13]. We design a safety admission controller that sits between the lightweight risk estimator and the strong guardrail, and that makes a joint decision based on request risk, tenant policy, remaining safety budget, current queue and load state, and the request's safety-stage deadline. The controller preserves a fail-closed invariant throughout: whenever tenant policy requires a strong check, the request either receives that check if capacity allows, or is rejected early if it cannot be completed in time—it is never silently forwarded to the generation model without the required check.

This paper makes the following contributions:

- **Safety capacity as a first-class gateway resource.** We separate physical provider capacity for a strong guardrail from a gateway-controlled safety budget, and study how limited strong-check capacity should be allocated across concurrent tenants and traffic conditions.
- **Tenant-aware safety admission control.** We show that the same risk score can warrant different actions for different tenants, and we design a scheduler that reserves scarce strong-check capacity for more sensitive tenants rather than treating all requests identically.
- **Deadline-aware, fail-closed admission.** We show that requests which require a strong check but cannot complete

it before their safety-stage deadline should be rejected early rather than queued indefinitely or, worse, allowed to bypass the check.

- **Empirical evidence of safety-resource exhaustion.** We show experimentally that the safety subsystem of a gateway can become overloaded even when total request throughput is held constant, purely because the fraction of risky requests in the traffic mix increases.

II. Related Work

A. Guardrails and Safety Enforcement

Safety research has developed learned guard models, moderation datasets, and benchmarks for identifying unsafe prompts, unsafe responses, jailbreaks, and over-refusal. Llama Guard provides an input/output safeguard model [3]; WildGuard broadens moderation across harmfulness, jailbreak, and refusal detection [2]; XSTest targets exaggerated safety behavior [1]; and AEGIS2.0 provides a broad safety-risk taxonomy and guardrail dataset [5]. Runtime frameworks such as NeMo Guardrails expose programmable enforcement rails [4], while managed services such as Amazon Bedrock ApplyGuardrail provide an external policy-enforcement API [7]. These systems primarily study what should be blocked or how a safety policy is enforced; they do not formulate the allocation of policy-required strong checks as a shared, budget-constrained resource across competing tenants.

B. Cost-Aware Routing and Cascades

A separate line of work reduces inference cost by routing requests among models or cascades. FrugalGPT adaptively combines model calls under cost–quality trade-offs [8], while RouteLLM learns per-request routing between stronger and weaker LLMs from preference data [9]. Their decision variable is which generation model or cascade should serve a request. Closer to our setting, recent guardrail work has explored cascaded detector architectures that selectively invoke a second-stage detector to improve moderation accuracy and computational efficiency [16]. Such work studies *when* to escalate safety processing to a specialized second-stage detector, but does not model contention for bounded second-stage capacity. Our setting is different: tenant policy may make a strong safety check mandatory, so the controller instead addresses *how scarce, already-triggered* strong-check capacity should be allocated across competing tenants under a bounded budget and per-request deadlines, preserving a fail-closed constraint rather than trading safety enforcement away for lower cost.

C. LLM Serving and Resource Scheduling

LLM serving systems study fairness, dynamic scheduling, tail latency, and SLO-constrained goodput for inference resources. Fairness in Serving Large Language Models introduces token-aware fair scheduling across clients [10]; Llumnix dynamically reschedules requests for load balance, priority, and SLO isolation [11]; DistServe optimizes serving capacity under TTFT and TPOT constraints [12]; and Sarathi-Serve studies the throughput–latency trade-off in batched LLM inference [13]. More broadly, Dominant Resource Fairness formalizes fair allocation of shared resources [14], and tail-latency work motivates deadline-aware control in large-scale services [15]. We borrow these systems concepts but apply them to a different resource and constraint set: strong-guardrail invocations have their own budget and queue, tenants have heterogeneous safety policies, and policy-required checks cannot be silently bypassed under overload.

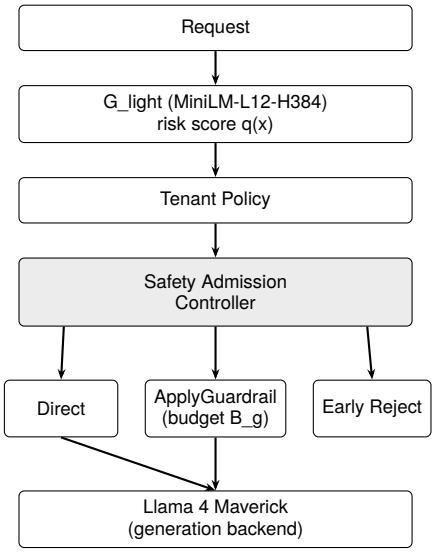


Fig. 1. Gateway architecture: lightweight risk estimator, tenant policy, safety admission controller, bounded strong-guardrail budget, and fail-closed routing to direct pass-through, ApplyGuardrail, or early rejection.

III. System Design

Fig. 1 shows the overall gateway architecture. An incoming request first passes through a lightweight risk estimator, G_{light}, implemented as a MiniLM-L12-H384-based estimator [6] that produces a continuous risk score $q(x)$ rather than a hard label. This score is combined with the requesting tenant's safety policy to determine the request's safety requirement: whether a strong check is optional, required, or waivable under specific conditions.

The Safety Admission Controller then makes the routing decision. It has three possible outcomes: direct forwarding to the generation model when no strong check is required or policy allows a waiver; routing to the strong guardrail—in our deployment, the managed Amazon Bedrock ApplyGuardrail API [7]—when capacity within the bounded safety budget B_g is available; or early rejection when the request requires a strong check but the controller determines that the check cannot be completed before the request's safety-stage deadline, or that insufficient B_g token-bucket capacity is available. Only requests that pass the strong guardrail, or that are exempted by policy, are forwarded to the generation backend, in our setup a managed Llama 4 Maverick endpoint.

We distinguish the gateway-controlled budget B_g from the provider's physical service capacity. B_g is an organizational

invocation budget enforced by the gateway; provider throughput or rate limits can impose additional constraints, but are not assumed to equal B_g.

The controller's admission decision is a function of five inputs: (i) the request's risk score from G_{light}; (ii) the requesting tenant's policy, which may mandate, waive, or conditionally require a strong check; (iii) the currently available fraction of the safety budget B_g; (iv) the gateway's current queue depth and load; and (v) the request's remaining time budget before its safety-stage deadline expires.

A. Fail-Closed Invariant

The controller enforces a single non-negotiable invariant: if tenant policy requires a strong check for a given request, then that request is resolved in exactly one of two ways—it receives the strong check if capacity is available, or it is rejected early if the check cannot be completed safely within the request's deadline. The controller never silently bypasses a policy-required guardrail to preserve throughput or reduce latency. This invariant is what distinguishes the design from throughput-first admission control: under overload, the system sheds load rather than weakening safety guarantees.

B. Tenant-Aware Scheduling

Because tenants may declare different safety sensitivity, an identical risk score $q(x)$ can imply different actions depending on which tenant issued the request. The scheduler therefore reserves a configurable share of strong-guardrail capacity for higher-sensitivity tenants rather than serving all tenants on a first-come, first-served or purely load-aware basis, which would let a high-volume, low-sensitivity tenant crowd out a low-volume, high-sensitivity tenant's access to required strong checks.

C. Formalization

We formalize the admission rule to make the controller's contract precise and reproducible. For request i from tenant t_i with risk score $q_i = G_{light}(x_i)$, let

$$r_i = f(q_i, \pi_{t_i}) \in \{0, 1\} \tag{1}$$

indicate whether tenant t_i's policy π_{t_i} requires a strong check for this request ($r_i = 1$) given its risk score. The controller assigns an action $a_i \in \{\textsc{direct}, \textsc{strong}, \textsc{reject}\}$. The fail-closed invariant is the constraint

$$r_i = 1 \implies a_i \in \{\textsc{strong}, \textsc{reject}\}, \tag{2}$$

i.e. a policy-required check is never resolved by DIRECT forwarding. Our implementation enforces the strong-check budget with parallel token buckets rather than a single global one: a shared bucket refills at rate $B_g(1 - \sum_c \rho_c)$ and each reserved sensitivity class c has its own bucket refilling at rate $\rho_c B_g$, every bucket capped at capacity b ($b = 1$ check in our experiments). Because the per-bucket rates sum to exactly B_g, the long-run strong-check rate is conserved at B_g; however, since the shared bucket and each reserved-class bucket can independently hold up to b tokens, the aggregate instantaneous burst is $(1 + C)b$ rather than b, where C is the number of reserved classes. Over any measurement window W, the controller admits at most

$$\sum_{i \in W} \mathbf{1}[a_i = \textsc{strong}] \;\le\; B_g\, T_W + (1 + C)b, \tag{3}$$

where B_g is the gateway-controlled strong-check rate budget (in checks per unit time; Section V) and T_W is the duration of window W. In the two-tenant configuration used in our experiments (Section V), only the sensitive tenant carries a reservation ($C = 1$), giving an aggregate bound of $B_g T_W + 2b$. Unused reservation in a class's bucket may be fully borrowed by another tenant once the reserving tenant is idle (no waiting or in-flight requests) — our implementation applies no additional fractional cap beyond each bucket's own capacity b — so idle reserved capacity is not wasted, while a class's reservation cannot be pre-empted while that tenant is active. Finally, request i is assigned $a_i = \textsc{reject}$ instead of STRONG whenever the controller's estimate of remaining queue wait plus strong-guardrail service time would exceed i's remaining safety-stage deadline, or whenever insufficient token-bucket capacity is available; otherwise $a_i = \textsc{strong}$ when $r_i = 1$, and $a_i = \textsc{direct}$ when $r_i = 0$ or policy grants a waiver. Our implementation uses a deliberately conservative wait estimate: zero when a token is available to the tenant, and otherwise one full refill interval of the tenant's bucket ($1/B_g = 2.5$ s for an unreserved tenant, $1/(\rho_c B_g)$ for a reserved class), ignoring partial refill. With the 600–800 ms deadlines used here this estimate always exceeds the deadline when no token is available, so in our experiments the deadline test reduces to "reject unless a token is available now"; Section VI-F discusses the consequence.

IV. Metrics

The primary performance metric is Safe SLO-Goodput, a safety-constrained analogue of SLO-oriented serving goodput [12]: the rate of requests that are simultaneously policy-compliant, ground-truth safe, served, and completed within their deadline:

$$G_{\text{safe}} = \frac{N(\text{policy-compliant} \wedge \text{safe} \wedge \text{served} \wedge \text{deadline met})}{T} \tag{4}$$

where $N(\cdot)$ counts qualifying requests over a measurement window of duration T, and ground-truth safety is determined by the frozen benchmark labels rather than by the strong guardrail's own decision. Rejected requests may preserve policy compliance but do not contribute to G_{safe} because they are not served. We report four supporting metrics alongside Safe SLO-Goodput, reported in the tables most relevant to each experiment: Unsafe Admission Rate (UAR), the fraction of ground-truth unsafe requests that are admitted to the generation backend, $\text{UAR} = N(\text{unsafe} \wedge \text{admitted})/N(\text{unsafe})$, which we decompose into admissions caused by G_{light} false negatives (routed DIRECT below threshold), strong-guardrail misses, and scheduler bypass; Required-Strong Coverage, the fraction of policy-required strong checks that were actually performed;

Policy Bypass Rate, the fraction of policy-required checks that were silently skipped; and safety-stage P95 latency. Unless stated otherwise, the deadline used in E1–E6 is the safety-stage / controller-path deadline (scheduler wait plus strong-guardrail latency); it is not a measured user-facing end-to-end generation SLO.

V. Experimental Setup

A. Benchmark Construction

We follow a generate-validate-freeze-replay methodology consistent with our prior guardrail evaluation work. Approximately 500 seed prompts are transformed into four risk classes: S0 (clear-safe), S1 (borderline-safe), S2 (direct-unsafe), and S3 (adversarial-unsafe). After validation, the frozen benchmark contains roughly 1.9K prompts. We additionally evaluate against two external benchmarks, XSTest [1] and WildGuardTest [2], to check that findings are not artifacts of our own prompt-generation process. E0a shows clear score separation on the frozen benchmark for binary safe/unsafe discrimination (S0–S1 as safe-negative, S2–S3 as unsafe-positive): AUROC 0.986, and transfers to the external sets with AUROC 0.811 on XSTest and 0.876 on WildGuardTest; full per-set precision/recall curves are reported in the artifact. Throughout, the MiniLM-based estimator G_{light} is used only as the gateway's risk-scoring component; it is never treated as ground truth for whether a prompt is actually safe or unsafe, which is instead determined by the frozen benchmark labels and the external test sets.

Each E1–E6 condition was repeated five times; we report the median across repetitions with the interquartile range [P25, P75] where space permits, and give ground-truth-unsafe counts pooled over repetitions alongside UAR.[1] The strong guardrail is a published (non-draft) Amazon Bedrock guardrail version with content filters (hate, insults, sexual, violence, misconduct) at high strength and prompt-attack detection on input, created from the released policy file; every strong-routed request in E1–E6 records the guardrail's verdict, and runs abort if a call returns none.[2] Unless otherwise stated, the main experiments use a frozen, gateway-controlled aggregate safety budget of $B_g = 0.4$ strong checks/s, implemented using the parallel token-bucket allocation described in Section III-C, with each bucket capped at one check. This is an organizational budget, not the provider's physical ApplyGuardrail capacity: E0b separately measures the provider's raw ApplyGuardrail throughput at approximately 101.7 RPS at concurrency 16 (P50/P95 141/231 ms), more than two orders of magnitude above B_g, with zero throttling up to concurrency 16 and 0.1% at 32; this makes provider-side throttling an unlikely explanation for the safety-resource pressure studied in E1–E6, which instead arises from the gateway-controlled budget by design.

Implementation parameters. The tenant configuration used in E1–E6 fixes a policy threshold τ (request requires a strong check when $q(x) \geq \tau$) and a safety-stage deadline (SLO) per tenant class: the majority tenant uses τ=0.75 and a 600 ms deadline, and the sensitive tenant uses a lower threshold τ=0.40 (routing more of its traffic to a strong check) and an 800 ms deadline. The sensitive tenant's reserved share is ρ=0.4 (40% of B_g); the majority tenant draws from the remaining shared pool. We fix $\rho = 0.4$ once, before the E2 tenant-mix sweep, and do not retune it across mixes. B_g here models an organizational safety-spend quota rather than a service-latency capacity limit, so the 600–800 ms deadlines should be read against queueing and strong-guardrail service time under that quota, not against B_g's refill interval directly. E1 and E3–E6 run the majority tenant only, retaining its τ and deadline from the same fixed configuration; E2 additionally introduces the sensitive tenant to study multi-tenant contention. We do not retune τ or the deadlines across experiments.

Baselines. We compare the proposed controller against three policies that share the same B_g token bucket and never bypass a required check. *Always-Strong* requires a strong check for every request, ignoring $q(x)$; it rejects requests whose deadline cannot be met and otherwise queues when the budget is exhausted. *Risk-only* requires a strong check when $q(x) \geq 0.5$ (a single global threshold, no tenant policy) and rejects when no token is available, with no deadline reasoning. *Load-aware* uses the same global threshold, rejects early when the estimated queue wait plus strong-check time exceeds the deadline, and otherwise queues, sharing capacity first-come, first-served across tenants (hence in proportion to volume). Queued requests time out after 2 s. Because Always-Strong and Load-aware use the same conservative wait estimate as the proposed controller (Section III-C), their deadline test also rejects whenever no token is available, and in E1–E4 fewer than 0.1% of their requests ever wait in the queue; in practice the three selective baselines therefore differ little in admission mechanics, and Risk-only and Load-aware behave almost identically. The proposed controller adds tenant-specific thresholds (τ=0.75 for the majority tenant versus the baselines' global 0.5) and per-tenant reservations. In single-tenant experiments the higher threshold means it requires fewer strong checks than the baselines, which we account for when interpreting E1, E3, and E4.

B. Experiment Sequence

We run three characterization experiments before the main study: E0a characterizes the MiniLM risk estimator's score distribution and separability across the four risk classes; E0b characterizes the latency and throughput of the managed ApplyGuardrail-style strong check in isolation; and E0c characterizes the latency and throughput of the Llama 4 Maverick generation backend in isolation. These establish baseline per-component behavior before the admission controller is intro-

[1]Code, configurations, guardrail policy definition, E0 characterization results, and full per-repetition results: https://github.com/taixingbi/guardrail-admission-control.

[2]The E0b–E6 numbers in this version come from a complete rerun against a guardrail re-created from the same released policy file in a new account, after we found that an earlier campaign had been run with an unset guardrail identifier and therefore never executed the strong check. G_{light} scores (E0a) and the generation-backend characterization (E0c) do not depend on the guardrail and are unchanged.

TABLE I
COMPONENT CHARACTERIZATION (E0). LATENCIES IN MS.

E0a: G_{light} (MiniLM-L12-H384), $\tau = 0.5$				
Set (n)	**AUROC**	**Recall**	**FPR**	**P50 / P95**
Frozen (1888)	0.986	0.956	0.034	524 / 619
XSTest (450)	0.811	0.375	0.048	522 / 628
WildGuardTest (1699)	0.876	0.720	0.105	661 / 1356
E0b: ApplyGuardrail (strong check)				
Concurrency	**RPS**	**P50**	**P95**	**Throttle**
1	5.25	181	254	0
16	101.67	141	231	0
32	110.17	147	806	0.1%
E0c: Llama 4 Maverick (generation)				
Concurrency	**RPS**	**TTFT P50**	**TTFT P95**	**E2E P95**
1	2.10	252	327	556
2	4.30	249	322	563
4	2.00	1094	6697	6838

duced. Table I summarizes them. G_{light} separates the frozen benchmark well but transfers imperfectly (XSTest recall 0.375 at $\tau = 0.5$), which is why it is used only to decide whether a strong check is needed, never as the authority. It is also not faster than the strong check: its endpoint P50 (524 ms) exceeds ApplyGuardrail's (141–181 ms at concurrency ≤ 16), so its role is cost reduction, not latency reduction. ApplyGuardrail sustains about 100 RPS before P95 degrades, whereas Maverick saturates at concurrency 2 (4.30 RPS); we set the gateway load to $R_{\text{gateway}} = 0.7 \times 4.30 = 3.01$ RPS.

To isolate scheduling effects from estimator and generation-backend variability, E1–E6 replay the frozen risk scores $q(x)$ produced during E0a rather than re-running G_{light} live, while still invoking the strong guardrail (ApplyGuardrail) live; the generation backend is not placed on the hot path of these experiments. We separately run an end-to-end check (proposed controller, $1.0B_g$, 40 s) with frozen $q(x)$, the live strong guardrail, and the live Maverick backend on the hot path: user-perceived end-to-end latency is P50/P95 555/703 ms, Safe SLO-Goodput is 1.775 RPS at the 600 ms deadline, 70% of admitted requests meet it, and 0 of 17 ground-truth unsafe requests reach generation.

The main study consists of six experiments. E1 (static safety-budget pressure) measures Safe SLO-Goodput and UAR as offered strong-check demand is held at 0.5, 1.0, and $1.5B_g$. E2 (multi-tenant contention) measures how strong-check coverage is distributed across tenants of different sensitivity under a fixed budget and varying tenant traffic mixes. E3 (dynamic safety demand) measures system behavior as the risky-request fraction changes over time within a session. E4 (safety-resource exhaustion) holds total gateway RPS constant while varying the suspicious-request fraction, isolating safety-budget pressure from changes in aggregate offered load. E5 (fail-open vs. fail-closed) compares the fail-closed controller against a fail-open baseline that bypasses required checks under budget pressure. E6 (deadline / early-rejection ablation) compares the full controller against a variant with early rejection disabled. E1–E4 compare all four policies; E5 and E6 vary the proposed controller only.

TABLE II
BASELINES AT $1.0B_g$ STATIC DEMAND (E1) AND UNDER A DYNAMIC MIX (E3); MEDIAN [P25, P75] OVER 5 REPS, UAR COUNTS POOLED

	E1 ($1.0B_g$)		**E3 (dynamic)**	
Policy	**G_{safe}**	**UAR**	**G_{safe}**	**UAR**
Always-Strong	0.18 [0.15, 0.20]	0/76	0.17 [0.17, 0.18]	1/874
Risk-only	2.55 [2.53, 2.58]	5/84	2.59 [2.57, 2.60]	32/816
Load-aware	2.48 [2.45, 2.48]	6/93	2.60 [2.58, 2.60]	39/810
Proposed	2.53 [2.48, 2.63]	5/79	**2.64** [2.62, 2.65]	34/816

VI. RESULTS

A. Baselines Under Static and Dynamic Load (E1, E3)

Table II compares the four policies. Always-Strong spends the entire budget on checks that are mostly unnecessary (only 6–24% of its strong-check occupancy is on requests with $q(x) \geq \tau$) and, because each live check takes ≈190 ms against a 0.4 checks/s budget, rejects 93–95% of all traffic (the high-strength guardrail additionally blocks 64% of the ground-truth-safe requests it checks in E1, mostly borderline S1 prompts); its UAR of zero is bought by refusing almost every request, and its Safe SLO-Goodput is roughly $14\times$ lower than any selective policy. The three selective policies are statistically close under static demand (E1: Proposed 2.70, 2.53, 2.40 vs. Risk-only 2.68, 2.55, 2.35 at 0.5, 1.0, $1.5B_g$). Under the dynamic mix of E3, in which offered strong demand moves $0.5 \rightarrow 0.9 \rightarrow 1.5 \rightarrow 0.6B_g$ over 480 s, Proposed attains the highest Safe SLO-Goodput and lowest reject rate (0.117 vs. 0.130–0.136); the gap is concentrated in the $1.5B_g$ overload phase (2.46 [2.36, 2.52] vs. 2.33 for both Risk-only and Load-aware), where it also starves fewer required checks (36 vs. 46). Most of this edge comes from the majority tenant's higher threshold (τ=0.75 vs. 0.5), which sends more low-risk requests direct and spends fewer tokens: the E6 variant that applies the global threshold to the proposed controller drops to 2.42 [2.33, 2.43], overlapping the baselines. The strong guardrail let through no examined unsafe request in E1 and at most 3 per policy across all E3 repetitions, so the residual UAR of the selective policies (0.04–0.05 in E3) consists almost entirely of G_{light} false negatives.

B. Tenant Isolation (E2)

Table III shows strong-check coverage for a minority sensitive tenant under two scheduling policies as the tenant mix is varied from 90:10 (majority:sensitive) down to 30:70. Under load-aware scheduling, which allocates strong-guardrail capacity in proportion to per-tenant request volume, the sensitive tenant's coverage falls as its share of traffic shrinks, dropping to 29.9% when it represents only 10% of total traffic. Tenant-aware scheduling, which reserves capacity for higher-sensitivity tenants regardless of volume, raises coverage to 65.5% at the same 90:10 mix, and continues to dominate load-aware coverage across every mix we tested, though the gap narrows as the sensitive tenant's own share of traffic grows. This result shows that a purely load-proportional allocation

of a shared safety resource systematically disadvantages low-volume, high-sensitivity tenants across the full range of tenant mixes, not only at the most favorable operating point, and that explicit tenant-aware policy—tenant-specific thresholds backed by a reservation (Section VI-F)—is needed to correct this. The extra coverage also reduces the sensitive tenant's unsafe admission rate UAR_B: tenant-aware scheduling admits 0.175, 0.140, and 0.122 of that tenant's ground-truth unsafe requests at 70:30, 50:50, and 30:70, versus 0.242, 0.232, and 0.206 under load-aware scheduling, with non-overlapping interquartile ranges; at 90:10 the difference (0.222 vs. 0.231) is within run-to-run noise. The UAR_B gain comes from the sensitive tenant's lower threshold (τ=0.40), not from the reservation: Section VI-F shows that UAR_B is unchanged when the reservation is removed ($\rho = 0$), while coverage drops. The risk-only baseline, which uses neither, has the highest UAR_B (0.231–0.294). Always-Strong attains the lowest UAR_B (0.01–0.05) only by checking 12–14% of B's required checks and rejecting the rest, which collapses overall Safe SLO-Goodput to about 0.1 RPS against 1.2–2.0 RPS for the selective policies.

C. Safety-Resource Exhaustion (E4)

We hold total gateway RPS constant at 3.01 and vary only the fraction of suspicious requests in the traffic mix from approximately 5% to 50% and back to 5%. Table IV reports offered strong-guardrail demand (relative to the safety budget B_g), Safe SLO-Goodput, and Required-Strong Coverage (Section IV) at each operating point. As the suspicious fraction rises from 5% to 50%, offered strong-guardrail demand rises from $0.38B_g$ to $3.76B_g$—nearly a $10\times$ increase—while Safe SLO-Goodput falls from 2.817 [2.800, 2.850] to 1.478 [1.461, 1.550] and Required-Strong Coverage drops from 0.67 to 0.23, even though total gateway offered load remains fixed at 3.01 RPS throughout. This isolates safety-budget pressure from changes in aggregate offered request rate; because the generation backend is not on the hot path of this experiment (Section V), we do not directly measure Maverick-side inference load here, but the fixed offered RPS rules out increasing aggregate request volume as the driver of the observed degradation. Returning the suspicious fraction to 5% recovers Safe SLO-Goodput to 2.850. This demonstrates that the safety subsystem can become an independent bottleneck on system throughput, distinct from and not explained by aggregate request volume—a failure mode that a purely volume-based admission controller would not detect. The exhaustion is a property of the workload, not of one policy: in the 50% phase all three selective policies are statistically tied (1.478–1.494 RPS, overlapping IQRs), and Always-Strong falls to 0.083. Over the whole run Proposed has the highest Safe SLO-Goodput (2.257 vs. 2.229 Load-aware, 2.183 Risk-only) but a slightly higher UAR (0.059 vs. 0.044–0.048); both follow from its higher majority-tenant threshold, which admits more requests directly, including more G_{light} false negatives. E4 therefore shows no safety advantage for any selective policy. Table V puts E3 and E4 side by side: Proposed's edge appears only in the moderate overload of E3 ($1.5B_g$), where its higher threshold leaves more of the budget for the checks it does require. In the extreme overload of E4 ($3.76B_g$) every selective policy saturates at the same coverage (0.22–0.23) and goodput, because any policy can complete only B_g checks per second.

D. Fail-Closed vs. Fail-Open (E5)

Table VI compares the fail-closed controller against a fail-open baseline that, under safety-budget pressure, silently forwards requests to generation without the policy-required strong check rather than rejecting them. At $1.5B_g$ offered demand, fail-open raises Safe SLO-Goodput by only 0.017 RPS (2.333 to 2.350) while bypassing 57% of policy-required strong checks and raising the Unsafe Admission Rate twelvefold, from 0.051 (11 of 201 unsafe requests, all G_{light} false negatives) to 0.611 (108 of 177). At $2.0B_g$ the pattern repeats: +0.067 RPS (2.200 to 2.267), 62% bypass, and UAR 0.095 to 0.636. The G_{light}-attributable component of UAR is the same in both modes (0.05–0.10); essentially the entire increase is scheduler-induced bypass ($\mathrm{UAR}_{\mathrm{bypass}} = 0.53$–$0.59$), which the fail-closed invariant eliminates by construction. Weakening mandatory checks under overload is therefore a poor systems trade-off even when evaluated purely on goodput: at most a 3% goodput gain for roughly 7–$12\times$ more unsafe admissions.

E. Deadline-Aware Early Rejection (E6)

Table VII compares the full controller against a variant with early rejection disabled, so that requests requiring a strong check are always queued rather than rejected when they cannot meet their deadline. Disabling early rejection performs 34% more strong-check work (29 to 39 calls over the measured window) with identical Safe SLO-Goodput, UAR (0.041), and reject rate, while raising safety-stage P95 latency from 178 ms (essentially one live guardrail call) to 2002 ms, which is the 2 s queue timeout: the magnitude of this gap is set by the timeout, not measured independently of it. The extra checks buy nothing: they block, after a long queue wait, requests that the full controller rejects at admission time because they cannot meet the deadline, and the requests that still cannot be served time out in the queue instead of being rejected immediately. Disabling the deadline estimate alone changes nothing, because with no token available the token check rejects the same requests (Section III-C); applying the global threshold instead of the tenant's (no tenant awareness) slightly lowers Safe SLO-Goodput (2.42 vs. 2.46) and adds two checks. Early rejection therefore prevents the controller from spending strong-guardrail capacity on requests that cannot ultimately count toward Safe SLO-Goodput, and prevents the safety-stage queue from collapsing under load.

F. Sensitivity to ρ and to the Deadline

We reran the E2 arrivals (same seeds) for the proposed controller with the sensitive tenant's reserved share $\rho \in \{0, 0.2, 0.6\}$ (Table VIII). Coverage of that tenant's required checks rises monotonically with ρ, from 45.7% to 67.2% at 90:10 and from 32.4% to 43.8% at 70:30, and largely saturates beyond $\rho = 0.2$ at 90:10. Overall Safe SLO-Goodput is

TABLE III
SENSITIVE-TENANT (B) STRONG-CHECK COVERAGE (POOLED OVER 5 REPS) AND UNSAFE ADMISSION RATE UAR_B (MEDIAN [P25, P75]) ACROSS TENANT MIXES (E2)

	Coverage of B's required strong checks				UAR_B			
Mix (A:B)	**Always-Str.**	**Risk-only**	**Load-aware**	**Tenant-aware**	**Always-Str.**	**Risk-only**	**Load-aware**	**Tenant-aware**
90:10	13.7%	31.3%	29.9%	**65.5%**	0.05 [0.00, 0.05]	0.29 [0.17, 0.30]	0.23 [0.16, 0.24]	0.22 [0.13, 0.25]
70:30	12.7%	30.7%	27.1%	**40.3%**	0.03 [0.02, 0.03]	0.23 [0.13, 0.32]	0.24 [0.22, 0.27]	0.18 [0.15, 0.18]
50:50	13.5%	26.9%	25.6%	**30.9%**	0.01 [0.01, 0.01]	0.26 [0.26, 0.27]	0.23 [0.21, 0.26]	0.14 [0.14, 0.18]
30:70	12.4%	24.1%	22.0%	**25.2%**	0.01 [0.01, 0.02]	0.23 [0.22, 0.25]	0.21 [0.20, 0.23]	0.12 [0.10, 0.14]
Overall G_{safe} at 90:10 / 30:70 (RPS)					0.11 / 0.10	1.82 / 1.21	1.76 / 1.19	1.98 / 1.21

TABLE IV
SAFETY-RESOURCE EXHAUSTION UNDER CONSTANT GATEWAY LOAD (E4, PROPOSED; MEDIAN OVER 5 REPS)

Suspicious Mix	**Gateway RPS**	**Strong Demand**	G_{safe}	**Req.-Strong Coverage**
5%	3.01	$0.38B_g$	2.817	0.67
50%	3.01	$3.76B_g$	1.478	0.23
5%	3.01	$0.38B_g$	2.850	0.72

TABLE V
SAFE SLO-GOODPUT (RPS) BY PHASE, ALL POLICIES (E3, E4; MEDIAN OVER 5 REPS). REQUIRED-STRONG COVERAGE IN PARENTHESES FOR THE OVERLOAD PHASE.

	E3: strong demand ($\times B_g$)				E4: suspicious mix		
Policy	0.5	0.9	1.5	0.6	5%	50%	5%
Always-Strong	0.19	0.15	0.16 (.12)	0.18	0.15	0.08 (.12)	0.17
Risk-only	2.74	2.58	2.33 (.39)	2.67	2.76	1.49 (.22)	2.73
Load-aware	2.72	2.59	2.33 (.40)	2.66	2.77	1.48 (.23)	2.79
Proposed	2.77	2.60	**2.46** (.43)	2.76	2.82	1.48 (.23)	2.85

TABLE VI
FAIL-CLOSED VS. FAIL-OPEN (E5, PROPOSED; MEDIAN [P25, P75] OVER 5 REPS, UAR COUNTS POOLED)

Demand	**Mode**	G_{safe}	**UAR**	**Bypass**
$1.5B_g$	Fail-closed	2.33 [2.20, 2.33]	0.051 (11/201)	0
	Fail-open	2.35 [2.35, 2.43]	0.611 (108/177)	0.57
$2.0B_g$	Fail-closed	2.20 [2.20, 2.22]	0.095 (20/226)	0
	Fail-open	2.27 [2.25, 2.28]	0.636 (138/215)	0.62

TABLE VII
EFFECT OF DISABLING DEADLINE-AWARE EARLY REJECTION (E6, $1.5B_g$; MEDIAN [P25, P75] OVER 5 REPS; SAFETY-STAGE P95)

Variant	G_{safe}	**Strong Calls**	**P95 (ms)**
Full	2.46 [2.36, 2.52]	29 [27, 30]	178 [178, 179]
No early rej.	2.46 [2.36, 2.52]	39 [38, 41]	2002 [2002, 2002]

flat in ρ (1.98–1.99 and 1.71–1.72 RPS), so the reservation shifts checks toward the sensitive tenant at no measurable aggregate cost. UAR_B does not move with ρ (0.17–0.22): its improvement over load-aware comes from the tenant-specific threshold, while the reservation buys coverage. Even at $\rho = 0$, coverage exceeds load-aware (45.7% vs. 29.9%) because the majority tenant's higher threshold (τ=0.75 vs. 0.5) leaves it with far fewer required checks (527 vs. 1,357 pooled across the E2 mixes), freeing shared budget for the sensitive tenant; the reservation adds a further 16–22 points at the 90:10 mix. The chosen $\rho = 0.4$ is therefore not a tuned optimum.

We also reran the E6 overload trace with the majority tenant's deadline set to 400, 800, and 1200 ms (600 ms in E6), five repetitions each. Every outcome of both variants is identical across the four deadlines: the full controller performs 29 checks with safety-stage P95 of 178–188 ms, and disabling early rejection performs 39 checks with P95 of 2002 ms and the same Safe SLO-Goodput (2.458) and UAR (0.041). The reason is the conservative wait estimate (Section III-C): when the bucket is empty the controller assumes a full 2.5 s refill, which exceeds every deadline in this range, so the same requests are rejected. The actual wait for the next token lies anywhere between 0 and 2.5 s, so a tighter estimate that accounts for partial refill would admit some requests at 1200 ms that it rejects at 400 ms. This sweep therefore shows that the benefit of rejecting rather than queueing is robust to the deadline, but it does not exercise the deadline as an independent control; evaluating a refill-aware estimate is future work.

VII. DISCUSSION

Taken together, these results support treating strong-guardrail capacity as a schedulable resource rather than as an unlimited service invoked whenever a per-request risk threshold is crossed. Tenant-aware allocation (E2) shows that fairness in a shared safety resource does not follow automatically from fairness in traffic volume, and requires explicit tenant-aware policy: tenant-specific thresholds free shared budget, and a reservation adds a further 16–22 points of coverage at the 90:10 mix. Safety-resource exhaustion (E4) shows that overload of the safety subsystem is a distinct bottleneck that emerges without an increase in aggregate request rate, and needs its own monitoring and control. The fail-open comparison (E5) shows that under budget pressure, the temptation to relax mandatory checks yields a poor safety-for-throughput trade. The early-rejection ablation (E6) shows that rejecting required checks that cannot start in time, rather than queueing them, is not merely a latency optimization but a mechanism that protects the safety queue itself from collapse; with our

TABLE VIII
SENSITIVITY TO THE SENSITIVE TENANT'S RESERVED SHARE ρ (PROPOSED, E2 ARRIVALS; COVERAGE POOLED, OTHERS MEDIAN OVER 5 REPS). $\rho = 0.4$ IS THE E2 CONFIGURATION.

	90:10			70:30		
ρ	Cov. B	UAR$_B$	G_{safe}	Cov. B	UAR$_B$	G_{safe}
0.0	45.7%	0.17	1.98	32.4%	0.17	1.71
0.2	62.1%	0.22	1.99	37.2%	0.17	1.72
0.4	65.5%	0.22	1.98	40.3%	0.18	1.71
0.6	67.2%	0.22	1.98	43.8%	0.18	1.72
Load-aware	29.9%	0.23	1.76	27.1%	0.24	1.63

conservative wait estimate this decision is driven by token availability rather than by the deadline value itself.

It is important to state precisely what the fail-closed invariant does and does not guarantee. It prevents scheduler-induced bypass of a policy-required strong check; it does not eliminate errors originating from the lightweight risk estimator or from the strong guardrail itself. In particular, the residual Unsafe Admission Rate of the proposed controller reflects false negatives from G_{light} (the dominant term in our runs) and occasional misses by the strong guardrail, and exceeds that of the always-strong baseline, which attains near-zero UAR only by rejecting more than 90% of all traffic under the same budget. The controller guarantees policy compliance conditional on the risk score $q(x)$, not ground-truth safety. Its UAR advantage in E2 comes from applying the sensitive tenant's stricter threshold, which the reservation then backs with capacity (Section VI-F), not from better risk estimation; under a single tenant (E1, E3, E4) the selective policies have statistically similar UAR, and we do not claim that budget-constrained scheduling improves overall safety accuracy there.

A limitation of this study is that the workload generator, while validated and frozen before experimentation, is synthetic; the four risk classes and the tenant-mix scenarios approximate but do not replicate a live production traffic distribution. The strong-guardrail and generation backends are also specific managed services, and absolute latency and throughput numbers may differ across providers, though we expect the qualitative findings—the existence of a distinct safety-resource exhaustion mode and the benefit of tenant- and deadline-aware admission—to generalize across guardrail providers with similar capacity constraints.

VIII. CONCLUSION

This paper argued that strong safety enforcement in multi-tenant LLM gateways should be treated as a schedulable shared resource, alongside compute and queueing capacity, rather than as an unlimited service triggered by a per-request risk threshold. We presented a safety admission controller that jointly reasons about request risk, tenant policy, available safety budget, queue load, and per-request deadlines, while preserving a fail-closed invariant that never silently bypasses a policy-required strong check. After characterizing the risk estimator on a validated benchmark and two external test sets, we evaluated the admission controller using frozen risk scores and showed that tenant-aware scheduling substantially improves strong-check coverage for minority sensitive tenants, that safety-resource exhaustion is a distinct bottleneck that emerges without an increase in aggregate request rate, that fail-open degradation trades a large increase in unsafe admissions for a small throughput gain, and that deadline-aware early rejection protects the safety queue while leaving useful throughput unchanged. We position this work as a systems contribution to LLM gateway design: guardrail capacity should be measured, budgeted, and scheduled with the same rigor as inference capacity, and it should never be silently weakened when the system is under load.

REFERENCES

[1] P. Röttger, H. Kirk, B. Vidgen, G. Attanasio, F. Bianchi, and D. Hovy, "XSTest: A test suite for identifying exaggerated safety behaviours in large language models," in *Proc. NAACL-HLT*, 2024, pp. 5377–5400.

[2] S. Han, K. Rao, A. Ettinger, L. Jiang, B. Y. Lin, N. Lambert, Y. Choi, and N. Dziri, "WildGuard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of LLMs," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[3] H. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Mao, M. Tontchev, Q. Hu, B. Fuller, D. Testuggine, and M. Khabsa, "Llama Guard: LLM-based input-output safeguard for human-AI conversations," arXiv:2312.06674, 2023.

[4] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in *Proc. EMNLP System Demonstrations*, 2023, pp. 431–445.

[5] S. Ghosh, P. Varshney, M. N. Sreedhar, A. Padmakumar, T. Rebedea, J. R. Varghese, and C. Parisien, "AEGIS2.0: A diverse AI safety dataset and risks taxonomy for alignment of LLM guardrails," in *Proc. NAACL-HLT*, 2025, pp. 5992–6026.

[6] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[7] Amazon Web Services, "ApplyGuardrail – Amazon Bedrock API Reference," AWS Documentation. [Online]. Available: https://docs.aws.amazon.com/bedrock/latest/APIReference/API_runtime_ApplyGuardrail.html

[8] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," *Transactions on Machine Learning Research*, 2024.

[9] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, "RouteLLM: Learning to route LLMs from preference data," in *Proc. ICLR*, 2025.

[10] Y. Sheng, S. Cao, D. Li, B. Zhu, Z. Li, D. Zhuo, J. E. Gonzalez, and I. Stoica, "Fairness in serving large language models," in *Proc. OSDI*, 2024, pp. 965–988.

[11] B. Sun, Z. Huang, H. Zhao, W. Xiao, X. Zhang, Y. Li, and W. Lin, "Llumnix: Dynamic scheduling for large language model serving," in *Proc. OSDI*, 2024, pp. 173–191.

[12] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *Proc. OSDI*, 2024, pp. 193–210.

[13] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov, and R. Ramjee, "Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve," in *Proc. OSDI*, 2024, pp. 117–134.

[14] A. Ghodsi, M. Zaharia, B. Hindman, A. Konwinski, S. Shenker, and I. Stoica, "Dominant resource fairness: Fair allocation of multiple resource types," in *Proc. NSDI*, 2011.

[15] J. Dean and L. A. Barroso, "The tail at scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013.

[16] M. Nagireddy, I. Padhi, S. Ghosh, and P. Sattigeri, "When in doubt, cascade: Towards building efficient and capable guardrails," in *Proc. AAAI/ACM Conf. on AI, Ethics, and Society (AIES)*, 2025.